generator

Generators is a function that provides sequence of elements.

So it is also an iterator.(list)

Storing the values one at a time rather than storing them altogether.

Generators are defined using the keyword ' yield'

Returns one at a single call and resume where it left off for the next call.

Raise the stopiteration automatically.

In [55]:
```
def hai():
    x=10
    while x>=0:
        print(x)
        x=x-1
```

In [56]:
```
hai()
```

```
10
9
8
7
6
5
4
3
2
1
0
```

In [52]:
```
def haii():
    x=10
    while x>=0:
        yield (x)
        x=x-1
```

In [53]:
```
haii()
```

Out[53]:
```
<generator object haii at 0x00000181B4F2AE40>
```

In [54]:
```
for i in haii():
    print(i)
```

```
10
9
8
7
6
5
4
3
2
1
0
```

In [9]:
```python
def simple():
    yield 1
    yield 2
    yield 3

# Driver code to check above generator function
for i in simple():
    print(i)
```

```
1
2
3
```

Decorators

A Python decorator is a function that takes in a function and returns it by adding some functionality.

A decorator is used to wrap a function

It gives new functionality without changing the code of original function source code A special type of function.

Decorator function return always another function

In [15]:
```python
def function(x):
    return x*2
```

In [16]:
```python
function(5)
```

Out[16]:
```
10
```

In [57]:
```python
def outer_function(x): #x=5
    def inner_function(x): #y=5
        return x * 2
    result = inner_function(x)
    return result
```

In [33]:
```python
def outer_function(x):
    def inner_function(y):
```

```
        return y * 2  # This should be y instead of result
    return inner_function(x)  # We need to pass x to inner_function
```

In [32]:
```
# Calling the functions
result = outer_function(5)  # Call outer_function with x=5
print(result)  # This will print the result of inner_function(5)
```

```
10
```

explanation

outer_function(5): This call invokes the outer_function with the argument x=5.

inner_function(y): Inside outer_function, inner_function is defined, which takes a parameter y.

inner_function(x): Within outer_function, inner_function is called with the argument x passed from outer_function.

return y * 2: inner_function returns the value of y multiplied by 2.

Finally, result = outer_function(5) stores the returned value (which is the result of inner_function(5)) in the variable result.

example 2

In [34]:
```
def check_odd_even(number):
    def is_even(num):
        print(number,'is even' if num % 2 == 0 else 'is odd')

    # Calling the inner function with the provided number
    is_even(number)

# Using the nested function
check_odd_even(10)
```

```
10 is even
```

example3

define a function to add 2 numbers

In [37]:
```
def add_numbers(a, b): # first function
    result = a + b
    return result
```

In [38]:
```
add_numbers(11,22)
```

Out[38]:
```
33
```

how can i combine the example 2 and 3

In [44]:
```python
def odd_even(result):
    def inner(*args): #This is where the actual wrapping of the original function happ
        answer = result(*args)  # Call the original function
        print(answer, 'is even' if result % 2 == 0 else 'is odd')
        return answer  # Return the result to maintain the behavior of the decorated f
    return inner

# Applying the decorator to the addition function
@odd_even
def add_numbers(a, b):
    result = a + b
    return result #result=97

# Using the decorated function
add_numbers(89, 8)  # Passing positional arguments directly
```

```
97 is odd
```

Out[44]:
```
97
```

explanation

Decorator Definition (odd_even):

odd_even is a decorator function that takes another function (result) as its argument. Inside odd_even, you define a nested function called inner. This function will wrap the original function result.

inner Function:

inner is a closure that takes any number of arguments (*args). *Inside inner, you call the original function (result) with the given arguments (result*(args)). You store the result of the original function call in a variable called answer. You print the result along with whether it's even or odd. Finally, you return the answer.

Decorator Usage (@odd_even):

You apply the odd_even decorator to the add_numbers function by using @odd_even syntax. This means that add_numbers will be passed to the odd_even function as an argument, effectively decorating add_numbers with the behavior defined in odd_even.

Decorated Function (add_numbers):

Inside the add_numbers function, you perform some operation (in this case, addition of two numbers a and b) and store the result in a variable called result. Then you return the result.

Using the Decorated Function:

You call the add_numbers function with arguments 89 and 8. This call is intercepted by the odd_even decorator, which prints whether the result of the addition is even or odd.

example 4

In [71]:
```python
def odd_even(func):
    def inner(*args): #wrapping the actual function -substraction (a,b)
        result = func(*args) # a*b  # Call the original function
        print(result, 'is even' if result % 2 == 0 else 'is odd')
    return inner
#When you write return inner, you're not actually calling the inner function;
#instead, you're returning a reference to the inner function itself.
@odd_even
def multiply(a, b):
    return a * b
 #value=30


@odd_even
def add(a, b):
    return a + b

@odd_even
def subtract(a, b):
    return a - b

# Using the decorated functions
multiply(5, 6)  # should print whether 5 * 6 is odd or even
add(7, 8)       # should print whether 7 + 8 is odd or even
subtract(10, 3) # should print whether 10 - 3 is odd or even
```

```
30 is even
15 is odd
7 is odd
```

example 5

In [47]:
```python
# Decorator for checking if the sum is odd or even
def check_odd_even(func):
    def inner(*args):
        result = func(*args)
        print("Sum is", 'even' if result % 2 == 0 else 'odd')
        return result
    return inner

# Decorator for checking if the result is a multiple of 10
def check_multiple_of_10(func):
    def inner(*args):
        result = func(*args)
        print("Result is", 'multiple of 10' if result % 10 == 0 else 'not multiple of
        return result
    return inner

# Function for adding two numbers
@check_odd_even
@check_multiple_of_10
def add(a, b):
    return a + b

# Using the add function
result = add(5, 6)
```

```
Result is not multiple of 10
Sum is odd
```

In [ ]: